

Moviestream

CLOPR1 PROJECT

Tristan De La Borda Baylon
BTS CLOUD COMPUTING | LYCÉE GUILLAUME KROLL

Contents

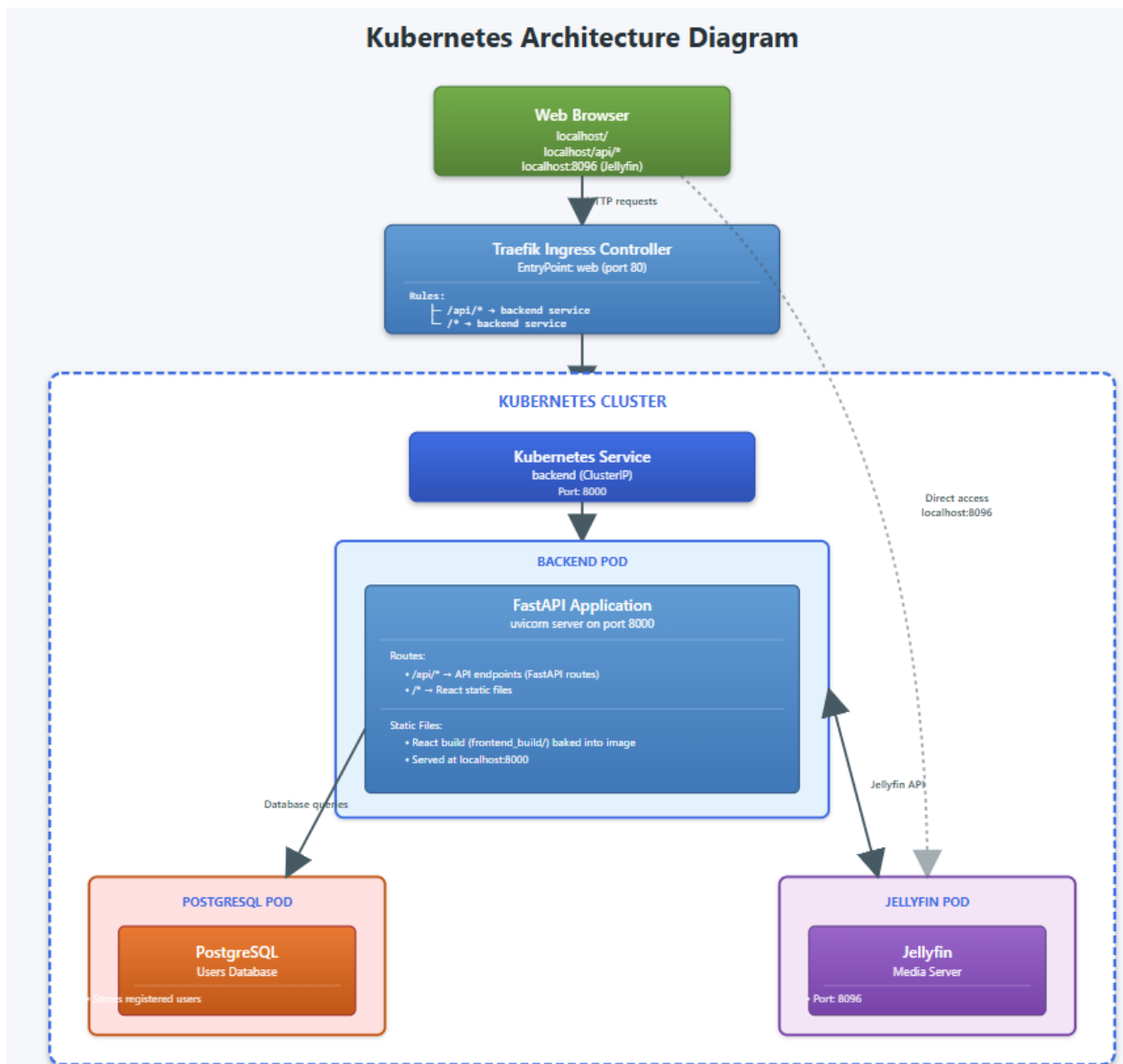
Contents	2
Introduction.....	3
Architecture Diagram	4
System Technology	4
Implementation	5
1. Environment Setup.....	5
1.1 Local Development Environment	5
1.2 Kubernetes Cluster Setup.....	5
2. Containerization.....	5
2.1 Backend API (FastAPI).....	5
Frontend (React)	6
Jellyfin (Media Server).....	6
PostgreSQL (database).....	7
Traefik (Ingress /Load Balancer).....	7
3. Kubernetes Deployment	7
3.1 Deployment and Pods.....	7
4. Ingress and Networking	8
4.1 Ingress Controller	8
5. Microservices	9
5.1 Secrets.....	9
Monitoring	10
1. Prometheus.....	10
2. Grafana	11
Conclusion	12

Introduction

In this project, we decided to create a Multimedia Streaming Platform webpage called Moviestream where users can browse, stream and manage video content. ore and serve movies, the platform integrates Jellyfin, an open-source media server. The application's frontend, backend, and database run in containers managed by Kubernetes, enabling easy deployment, scaling, and management.

This project provides practical experience in containerization, orchestration, cloud deployment, and monitoring using tools like Prometheus and Grafana, while demonstrating how a modern multimedia streaming platform can be built and maintained.

Architecture Diagram



System Technology

Component	Description	Technology
Frontend Service	Web interface for browsing and streaming content	React (dockerized)
Backend API Service	Handles authentication	FastAPI/python
Database service	Stores user data	PostgreSQL
Jellyfin Media server	Stores and streams	Jellyfin (dockerized)

Ingress Controller	Routes external HTTP/HTTPS traffic	Traefik
Monitoring Stack	Collects and visualized metrics	Prometheus + Grafana
Cloud Infrastructure		DigitalOcean kubernetes
		Kubernetes secret
		ConfigMaps

Implementation

1.Environment Setup

1.1 Local Development Environment

1.2 Kubernetes Cluster Setup

2.Containerization

2.1 Backend API (FastAPI)

For the backend of the application we used FastAPI, which is a modern, high-performance Python web framework designed for building APIs.

It is responsible for:

- Handling client HTTP requests
- Managing authentication and authorization
- Interacting with the PostgreSQL database
- Serving the frontend static files (React production build)
- Acting as the central API layer between the frontend, database, and external services (e.g. Jellyfin)

The backend runs inside a Docker container and is deployed as a Kubernetes Deployment, making it scalable, portable, and isolated from the host system.

(backend:dockerfile image)

Frontend (React)

The frontend of the application is developed using **React**, a JavaScript library for building interactive user interfaces.

It is responsible for:

- Displaying the user interface (UI)
- Display and browse through the movie library
- Streaming Movies

The frontend is built using Vite for fast development and optimized production builds. In production, the frontend is served as static files by the FastAPI backend, allowing a single entry point for the application.

(command to run it)

(frontend image)

Jellyfin (Media Server)

Jellyfin is an open-source media server used in this project to provide video streaming services.

It allows users to stream media content (movies, series, etc.) over the network using a web interface or compatible clients.

In this architecture, Jellyfin runs as a containerized service inside Kubernetes, separate from the backend and frontend application logic.

Jellyfin is responsible for:

- Hosting and streaming media content
- Providing a dedicated web interface for media playback
- Managing media libraries
- Handling transcoding and streaming protocols

(pic of the server and /movie)

PostgreSQL (database)

PostgreSQL is the relational database management system (RDBMS), which we used in this project to store users data. It serves as the persistent data layer for the FastAPI backend and handles all user-related information such as authentication credentials.

PostgreSQL is responsible for:

- Storing user accounts (email, password hashes)
- Supporting authentication and authorization
- Ensuring data consistency and integrity
- Persisting data across container restarts

(picture of the postgresql)

Traefik (Ingress /Load Balancer)

3.Kubernetes Deployment

3.1 Deployment and Pods

The following services are deployed as individual Pods within the Kubernetes cluster:

- Backend API (FastAPI)
- PostgreSQL Database
- Jellyfin Media Server
- Traefik Ingress Controller

Each Pod contains one container built from a Docker image specific to the service.

(Pictures of the pods)

3.2 Persistent Volumes

Persistent storage is a critical requirement for this project, as certain services must retain data even when Pods are restarted, rescheduled, or redeployed. To achieve this, Kubernetes Persistent Volumes (PVs) and Persistent Volume Claims (PVCs) are used.

In this project, persistent storage is required for:

- Database data (PostgreSQL)
- Media files and metadata (Jellyfin)

(pvc.yaml)

PostgreSQL Data PVC

- Stores:
 - User records
 - Password hashes
 - Database schemas and indexes
- Mounted into the PostgreSQL container
- Ensures database data is preserved across restarts

Jellyfin Media Library PVC

- Stores:
 - Media files (movies and series)
 - Jellyfin configuration
 - Playback metadata
- Mounted into the Jellyfin container
- Prevents data loss when Jellyfin Pods are recreated

4. Ingress and Networking

4.1 Ingress Controller

4.2 Ingress Rules

5. Microservices

5.1 Secrets

Kubernetes Secrets are used to store and manage sensitive configuration data securely.

Secrets ensure that confidential information is not exposed in application code or container images.

Secrets are used to store:

- Database passwords
- Database user credentials
- JWT signing secrets
- API keys (if applicable)

These values are required by the backend but must remain protected.

(show secrets)

5.2 ConfigMaps

ConfigMaps are used to store non-sensitive configuration data required by application services.

This approach allows configuration values to be modified without rebuilding container images.

ConfigMaps are used to manage:

- Application configuration values
- API endpoint URLs
- Environment-specific settings
- Service-level parameters

These values do not contain sensitive information and can safely be stored in plain text.

Show config

Monitoring

1. Prometheus

So, for the initial setup and installation of Prometheus the prerequisites are we have a virtual machine that has barebones Linux installation on it. To start off we will be immediately needing to install docker on the machine with the following commands that can be found on the following site: <https://docs.docker.com/engine/install/ubuntu/>

After following these steps which we have outlines many times I don't think it needs to be repeated we shall skip to the creation of the docker compose file that will illustrate the build of our Prometheus and later Grafana template.

For our dockercompose.yaml file this is what we have inside the file (Ignoring the parts related to Grafana we shall glance over those sections later):

```
prometheus > docker-compose.yaml
1  services:
2    prometheus:
3      image: prom/prometheus
4      container_name: prometheus
5      command:
6        - '--config.file=/etc/prometheus/prometheus.yml'
7      ports:
8        - 9090:9090
9      restart: unless-stopped
10     volumes:
11       - /home/user/prometheus/config/prom_etc:/etc/prometheus
12       - /home/user/prometheus/config/prom_data:/prometheus
13     user: "0:0"
14
15     grafana:
16       image: grafana/grafana:latest
17       container_name: grafana
18       ports:
19         - "3000:3000"
20       volumes:
21         - /home/user/prometheus/config/grafana:/var/lib/grafana
22       restart: unless-stopped
23       user: "0:0"
24
```

Here we outline what we want docker to create.

Going line by line we will first define the services in this case Prometheus, specifying the image we want and naming the container respectively for later use.

Then within the container we shall run the command for the config file found in the documentation template for Prometheus dockercompose.yaml file on the Prometheus docker GitHub page.

We shall open ports 9090 and bind it to 9090 on the internal to the bridged network for access to the container.

The container will restart if stopped thus ensuring longevity, then we will be making two directories in the container /etc/Prometheus and /Prometheus which will be binded to my local machines file prom_data and prom_etc for configuration later if needed.

Once we have this all setup we will simply run *"Docker compose up -d"* within the docker compose directory and this will deploy the Prometheus container.

2. Grafana

For the first part we used docker compose to deploy the Prometheus container we shall be using that exact docker compose file from earlier to deploy another container containing Grafana.

Let's look over the compose commands for Grafana:

```
grafana:
  image: grafana/grafana:latest
  container_name: grafana
  ports:
    - "3000:3000"
  volumes:
    - /home/user/prometheus/config/grafana:/var/lib/grafana
  restart: unless-stopped
  user: "0:0"
```

Pretty straight forward we will be pulling the latest Grafana image for the container.

Binding port 3000 to 3000 on the internal and bridged network.

Binding a volume `/home/user/Prometheus/config/grafana` from our local machine to the internal container Grafana folder for access to the internal files for alteration later if needed.

Then to maintain the container isn't stopped it shall restart always.

Running "*Docker compose up -d*" will deploy both the containers in the compose file which in this case is our Prometheus and Grafana.

The following output should show the correct functionality is now present and containers are launched:

```
user@Blade-XV:~/prometheus$ docker compose up -d
[+] up 2/2
✓ Container grafana    Running
✓ Container prometheus Running
user@Blade-XV:~/prometheus$
```

From this step we shall navigate to the web portal and type the IP suffixed by the Port and this should take us to the Prometheus and Grafana web portal.

Conclusion